

# **Protocolos de Comunicación**

## **Trabajo Práctico Especial**



### **Grupo 15**

#### **Integrantes:**

- Mateo López Badias
  - Julia Priolo
  - Olivia Rodríguez
  - Sofía Inés Ratto

**Fecha de entrega: 14/07/2026**

|                                                                                              |           |
|----------------------------------------------------------------------------------------------|-----------|
| <b>1. Descripción detallada de los protocolos diseñados y aplicaciones desarrolladas....</b> | <b>3</b>  |
| 1.1 Arquitectura general.....                                                                | 3         |
| 1.2 Protocolo SOCKS5.....                                                                    | 3         |
| 1.3 Resolución de nombres y fallback multi-IP.....                                           | 4         |
| 1.4 Gestion de usuarios.....                                                                 | 5         |
| 1.5 Métricas.....                                                                            | 5         |
| 1.6 Registro de acceso.....                                                                  | 5         |
| 1.7 Protocolo de monitoreo y configuración.....                                              | 6         |
| 1.8 Cliente de monitoreo.....                                                                | 8         |
| 1.9 Argumentos de línea de comandos.....                                                     | 8         |
| 1.10 Graceful shutdown.....                                                                  | 9         |
| <b>2. Problemas encontrados durante el diseño y la implementación.....</b>                   | <b>9</b>  |
| <b>3. Limitaciones de la aplicación.....</b>                                                 | <b>9</b>  |
| <b>4. Posibles extensiones.....</b>                                                          | <b>10</b> |
| <b>5. Conclusiones.....</b>                                                                  | <b>10</b> |
| <b>6. Ejemplos de prueba.....</b>                                                            | <b>10</b> |
| <b>7. Guía de instalación detallada y precisa.....</b>                                       | <b>11</b> |
| <b>8. Instrucciones para la configuración.....</b>                                           | <b>12</b> |
| <b>9. Ejemplos de configuración y monitoreo.....</b>                                         | <b>12</b> |
| <b>10. Documento de diseño del proyecto.....</b>                                             | <b>12</b> |

# 1. Descripción detallada de los protocolos diseñados y aplicaciones desarrolladas

Este informe documenta la implementación de un servidor proxy SOCKS5 con autenticación usuario/password, desarrollado en C sobre una arquitectura de I/O no bloqueante multiplexada con `pselect`. Además del proxy, se implementó un protocolo simple de administración y monitoreo en tiempo real, un registro de accesos y un cliente de línea de comandos para operar dicho protocolo.

## 1.1 Arquitectura general

El servidor corre en un único proceso y, salvo por la resolución de nombres, en un único hilo de ejecución. La concurrencia se apoya en un selector generico (`src/utils/selector.c`) que multiplexa múltiples file descriptors usando `pselect()`.

El selector mantiene una tabla de file descriptors registrados, sus intereses de lectura/escritura, handlers asociados (`handle_read`, `handle_write`, `handle_block`, `handle_close`) y un puntero opaco de estado por fd. En el servidor, ese estado es un `client_state_t *` para conexiones SOCKS5 o un `management_state_t *` para conexiones de management.

En el mismo loop de eventos se registran dos sockets pasivos no bloqueantes:

- socket SOCKS5, puerto por defecto `1080`;
- socket de management, puerto por defecto `8080`.

Cada conexión aceptada se registra con su propio estado, lo que permite atender múltiples conexiones concurrentes sin crear un hilo por cliente. Por estar basado en `select/pselect`, el límite de descriptors queda acotado por `FD_SETSIZE` y por la capacidad interna del selector.

El repositorio incluye módulos genéricos provistos por la cátedra, como un motor de máquinas de estado (`stm.c`), un parser por tabla de transiciones (`parser.c/parser_utils.c`) y un buffer genérico (`buffer.c`). En la implementación actual del servidor SOCKS5 estos módulos no forman parte del flujo principal: el parsing de SOCKS5 se realiza de forma manual e incremental sobre buffers planos asociados a cada `client_state_t`. Esta decisión permitió mantener control explícito sobre las etapas GREETING, AUTH, REQUEST, RESOLVING, CONNECTING, RELAY y CLOSING. Los módulos genéricos quedan disponibles en el repositorio y cubiertos por tests unitarios de utilidades.

La máquina de estados SOCKS5 se implementa mediante el enum `client_stage_t`. El despacho de eventos de cliente se concentra en `client_session.c`,

mientras que la resolución DNS y la conexión al destino se separaron en `target_connection.c`. El manejo de autenticación RFC1929 se aisló en `auth.c` y el protocolo de management en `management.c`.

## 1.2 Protocolo SOCKS5

La conexión SOCKS5 recorre los siguientes estados:

GREETING → AUTH → REQUEST → RESOLVING → CONNECTING → RELAY → CLOSING

AUTH y RESOLVING no siempre se atraviesan: AUTH depende del método negociado y RESOLVING solo se usa para destinos FQDN.

En el greeting se valida `VER = 0x05`, se leen los métodos ofrecidos por el cliente y se responde con un método soportado o con `0xFF` si no hay método aceptable. Actualmente se prioriza `USERNAME/PASSWORD` si el cliente lo ofrece. Si no lo ofrece y no hay usuarios configurados, acepta `NO AUTH`.

La autenticación RFC 1929 parsea:

`VER(1) | ULEN | UNAME | PLEN | PASSWD`

El servidor acumula bytes hasta tener el mensaje completo y valida usuario/password contra la tabla de usuarios en memoria.

El request soporta solamente el comando `CONNECT`. No se implementan `BIND` ni `UDP ASSOCIATE`; para esos comandos se responde `COMMAND_NOT_SUPPORTED`. Se aceptan destinos IPv4, IPv6 y FQDN. La respuesta de `CONNECT` usa siempre `ATYP = IPv4` y `BND.ADDR/BND.PORT = 0.0.0.0:0` como placeholder.

Una vez conectada la salida al destino, el servidor entra en `RELAY` y reenvía datos en ambos sentidos con buffers independientes de 4 KiB:

- `c2t_buffer`: cliente -> destino;
- `t2c_buffer`: destino -> cliente.

Los intereses de lectura/escritura de cada fd se actualizan según el estado de cada buffer, respetando lecturas y escrituras parciales de TCP.

Los errores de SOCKS5 se derivan de validaciones del request, errores de `connect()` y errores de `getaddrinfo()`, usando códigos como `GENERAL_FAILURE`, `NETWORK_UNREACHABLE`, `HOST_UNREACHABLE`, `CONNECTION_REFUSED`, `TTL_EXPIRED`, `COMMAND_NOT_SUPPORTED` y `ADDRESS_TYPE_NOT_SUPPORTED`.

## 1.3 Resolución de nombres y fallback multi-IP

Para evitar bloquear el hilo principal ante un FQDN, se crea un hilo dedicado por resolución DNS. Ese hilo ejecuta `getaddrinfo()` y, al finalizar, despierta al selector mediante `selector_notify_block()`, que usa una señal configurada (`SIGALRM`) para interrumpir el `pselect()`.

El hilo secundario no realiza I/O de datos; solo resuelve nombres y entrega el resultado al hilo principal. Si el cliente se desconecta mientras la resolución sigue pendiente, el estado se desacopla del `resolver_job` para evitar un use-after-free.

Cuando `getaddrinfo()` devuelve varias direcciones, el servidor intenta conectarse secuencialmente a cada una hasta que alguna tiene éxito.

## 1.4 Gestion de usuarios

Los usuarios SOCKS5 se almacenan en una tabla estática en memoria, con capacidad máxima `USERS_MAX = 10`. La tabla se inicializa con argumentos `-u user:pass` y puede modificarse en caliente desde management con `ADDUSER` y `DELUSER`.

No hay sincronización explícita sobre esta tabla porque tanto la autenticación SOCKS5 como el protocolo de management corren en el hilo principal del selector. Los hilos de DNS no acceden a los usuarios.

## 1.5 Métricas

El servidor mantiene métricas volátiles en memoria:

- conexiones históricas
- conexiones concurrentes activas
- conexiones exitosas hacia el destino
- conexiones fallidas hacia el destino
- bytes transferidos cliente -> destino
- bytes transferidos destino -> cliente
- bytes totales transferidos

Las métricas se actualizan durante aceptación, cierre, resultado del `CONNECT` y cada `send()` del relay. También se imprimen por stdout al finalizar el servidor.

## 1.6 Registro de acceso

Cada conexión que llega a intentar un `CONNECT` genera una línea en `access.log`:

```
<timestamp ISO-8601 UTC> user=<usuario|anonymous> dest=<host> port=<puerto>  
status=<OK|FAIL>
```

Ejemplo:

```
2026-07-11T14:45:42Z user=alice dest=127.0.0.1 port=29201 status=OK
```

El log registra accesos exitosos y fallidos, evita duplicados por conexión mediante un flag en `client_state_t` y no registra passwords. Este mecanismo permite identificar que usuario SOCKS5 se conectó a que destino y cuando.

## 1.7 Protocolo de monitoreo y configuración

Para la comunicación entre el cliente de administración y el servidor se definió un protocolo simple de tipo request-response sobre TCP. El protocolo fue desarrollado específicamente para este proyecto con el objetivo de permitir la consulta de métricas y la administración de usuarios SOCKS5 en tiempo de ejecución, sin reiniciar el servidor.

Sus principales características son:

- Transporte: TCP
- Dirección por defecto: `127.0.0.1`
- Puerto por defecto: `8080`
- Codificación: texto UTF-8 restringido al subconjunto ASCII para comandos, claves y separadores
- Modelo: request-response
- Ciclo de vida: una conexión TCP por comando
- Autenticación propia: no implementada

En cada operación, el cliente establece una conexión TCP con el servidor y envía una única línea que representa el comando solicitado. El servidor procesa la petición, responde con cero o más líneas en el formato `clave=valor` y, una vez finalizado el envío de la respuesta, cierra la conexión. Debido a este diseño, cada solicitud es independiente y no se mantiene estado de sesión asociado a cada conexión de management. Sí existe estado global del servidor, como la tabla de usuarios y las métricas, que puede ser consultado o modificado por distintos comandos.

Los mensajes no serializan estructuras binarias. Se eligió una serialización textual porque el volumen esperado del canal de monitoreo es bajo, los comandos son administrativos, y el formato puede inspeccionarse fácilmente con herramientas simples durante pruebas. Todos los números se codifican en base decimal sin separadores de miles. Las líneas terminan con `LF (\n)`; el servidor tolera requests terminados en `CRLF` porque descarta `\r/\n` al procesar el comando.

Gramática:

```
request = command [ SP arg1 [ SP arg2 ] ] LF
```

```
command = "STATS" | "METRICS" | "USERS" | "ADDUSER" | "DELUSER" | "HELP" |  
"QUIT"
```

Los comandos son case-insensitive. Los argumentos se separan por uno o más espacios o tabs. La implementación valida estrictamente la cantidad de argumentos en los comandos que modifican usuarios: ADDUSER y DELUSER. Para los comandos sin argumentos, los clientes compatibles no deben enviar parámetros adicionales, aunque la implementación actual toma únicamente el comando principal. En la implementación actual, el request completo debe entrar en el buffer de management de 1024 bytes.

Comandos:

| Comando         | Argumentos    | Descripción                        |
|-----------------|---------------|------------------------------------|
| STATS / METRICS | ninguno       | Devuelve métricas actuales         |
| USERS           | ninguno       | Lista usuarios SOCKS5 configurados |
| ADDUSER         | <user> <pass> | Agrega un usuario                  |
| DELUSER         | <user>        | Elimina un usuario                 |
| HELP            | ninguno       | Lista comandos soportados          |
| QUIT            | ninguno       | Responde ok=bye                    |

Formato de STATS:

```
historical_connections=<n>
concurrent_connections=<n>
successful_connections=<n>
failed_connections=<n>
bytes_transferred=<n>
bytes_client_to_target=<n>
bytes_target_to_client=<n>
```

Formato de USERS:

```
users_count=<n>
user=<nombre1>
user=<nombre2>
```

Formato de ADDUSER:

```
adduser=<ok|exists|full|invalid>
users_count=<n>
```

Formato de DELUSER:

```
deluser=<ok|not_found|invalid>
users_count=<n>
```

Formato de **HELP**:

```
commands=STATS,METRICS,USERS,ADDUSER,DELUSER,HELP,QUIT
```

Errores:

```
error=empty_command
error=unknown_command
error=usage_ADDUSER_user_pass
error=usage_DELUSER_user
```

Los clientes compatibles deben leer hasta EOF para obtener la respuesta completa. El orden de las líneas de cada respuesta es estable y está documentado arriba; para **USERS**, puede haber cero o más líneas **user=<nombre>** luego de **users\_count=<n>**.

## 1.8 Cliente de monitoreo

El binario **bin/client** abre una conexión TCP bloqueante al servicio de management, envía el comando indicado por línea de comandos y escribe la respuesta en stdout.

Uso:

```
./bin/client [-L management_addr] [-P management_port] [comando]
```

Si no se indica comando, envía **STATS**.

Ejemplos:

```
./bin/client -P 8089 STATS
./bin/client -P 8089 USERS
./bin/client -P 8089 ADDUSER pablito pass1234
./bin/client -P 8089 DELUSER pablito
./bin/client -P 8089 HELP
```

El cliente resuelve el host de management con **getaddrinfo()** y prueba las direcciones devueltas hasta poder conectarse.

## 1.9 Argumentos de línea de comandos

Servidor:

| Opción    | Descripción             | Default |
|-----------|-------------------------|---------|
| <b>-h</b> | Muestra ayuda y termina | -       |



|                  |                                            |           |
|------------------|--------------------------------------------|-----------|
| -v               | Muestra versión y termina                  | -         |
| -l <addr>        | Dirección de bind del proxy SOCKS5         | 0.0.0.0   |
| -p <port>        | Puerto del proxy SOCKS5                    | 1080      |
| -L <addr>        | Dirección de bind del management           | 127.0.0.1 |
| -P <port>        | Puerto del management                      | 8080      |
| -u <user>:<pass> | Usuario SOCKS5 inicial, repetible hasta 10 | ninguno   |

Cliente:

| Opción    | Descripción              | Default   |
|-----------|--------------------------|-----------|
| -h        | Muestra ayuda y termina  | -         |
| -L <addr> | Dirección del management | 127.0.0.1 |
| -P <port> | Puerto del management    | 8080      |

### 1.10 Graceful shutdown

El servidor maneja **SIGINT** y **SIGTERM**.

Ante la primera señal:

- Deja de aceptar nuevas conexiones SOCKS5 y management;
- Desregistra y cierra los sockets pasivos;
- Mantiene activas las conexiones SOCKS5 ya aceptadas;
- Espera hasta que no queden conexiones activas.

Ante una segunda señal, fuerza la salida del bucle principal y descarta las conexiones activas restantes. Al finalizar imprime un resumen de métricas y libera el selector.

## 2. Problemas encontrados durante el diseño y la implementación

Durante el desarrollo aparecieron varios problemas relacionados con el modelo de concurrencia elegido, la naturaleza de TCP como stream y la necesidad de mantener el hilo principal del servidor siempre disponible para atender múltiples conexiones.

### 2.1 Coordinación entre resolución DNS y ciclo de vida del cliente

Uno de los primeros problemas fue como resolver nombres de dominio sin bloquear al resto del servidor. La función `getaddrinfo()` puede demorar un tiempo no acotado, por ejemplo si hay latencia de red, servidores DNS lentos o timeouts. Si se ejecutaba directamente en el hilo principal, todo el selector quedaba detenido hasta que terminara la resolución, afectando también a clientes que no tenían relación con esa consulta.

Para resolverlo, se decidió ejecutar cada resolución DNS en un hilo auxiliar. Ese hilo recibe un `resolver_job`, llama a `getaddrinfo()` y luego notifica al selector cuando el resultado está disponible. El problema adicional fue que la conexión del cliente podrá cerrarse antes de que el hilo de DNS terminará. En ese caso, si el hilo secundario conservaba una referencia directa a `client_state_t`, podía terminar accediendo a memoria ya liberada.

La solución fue desacoplar explícitamente el ciclo de vida del cliente y del trabajo de resolución. El estado del cliente mantiene una referencia al `resolver_job`, y el `resolver_job` mantiene a su vez una referencia al estado del cliente sólo mientras este siga vivo. Si el cliente se desconecta durante la resolución, se llama a una rutina de desacople que deja el `resolver_job` sin estado asociado. Cuando el hilo DNS termina, verifica si el cliente sigue existiendo antes de notificar. Si ya no existe, libera sus propios recursos sin tocar el estado del cliente.

## 2.2 Despertar `pselect()` desde un hilo secundario

Al mover DNS a un hilo auxiliar apareció otro problema: el hilo principal podía estar bloqueado dentro de `pselect()` esperando eventos de sockets, mientras el hilo DNS terminaba su trabajo. Si no había ningún evento de red nuevo, el selector podía no enterarse inmediatamente de que la resolución había finalizado.

La solución fue usar el mecanismo de notificación provisto por el selector: el hilo secundario encola el resultado del trabajo bloqueante y despierta al hilo del selector mediante una señal configurada. El servidor usa `pselect()` justamente porque permite esperar eventos de file descriptors y, al mismo tiempo, manejar señales con una máscara controlada. Esto evita la clásica carrera entre "verifico si hay trabajo pendiente" y "me bloqueo esperando eventos".

De esta forma, cuando `getaddrinfo()` termina, el hilo principal despierta, procesa el evento `handle_block` asociado al cliente y continua el flujo normal: si la resolución fue exitosa intenta conectar con alguna de las direcciones obtenidas; si fallo, responde al cliente con un código SOCKS5 apropiado.

La decisión mantiene el modelo central del servidor: solo el hilo principal manipula sockets de clientes, sockets destino y estructuras principales de conexión. El hilo secundario solo resuelve nombres y notifica.

## 2.3 Manejo de lecturas parciales en el handshake SOCKS5

TCP es un protocolo de stream: no preserva los límites de los mensajes de aplicación. Por eso, aunque el cliente envíe un mensaje SOCKS5 completo, el servidor puede recibirlo partido en varios `recv()`. También puede ocurrir lo contrario: que lleguen varios bytes juntos y haya que interpretar sólo la parte correspondiente al estado actual.

Esto afectó especialmente al greeting, a la autenticación username/password y al request `CONNECT`, porque todos tienen campos de longitud variable o tamaños que recién se conocen después de leer algunos bytes iniciales. Si el servidor intentaba parsear inmediatamente cada `recv()` como si trajera un mensaje completo, podía rechazar conexiones válidas o leer fuera del rango disponible.

La solución fue acumular bytes en `read_buffer` y mantener un contador `read_len`. Cada estado verifica primero si hay suficientes bytes para interpretar el mensaje. Por ejemplo, en autenticación se espera a tener `VER`, `ULEN`, el usuario completo, `PLEN` y la password completa antes de validar credenciales. En requests con FQDN, se espera a tener el largo del dominio y luego los bytes del dominio más el puerto.

Con esto, el parser manual del servidor tolera fragmentación natural de TCP y solo avanza de estado cuando el mensaje está completo y validado.

## 2.4 Fallback entre múltiples direcciones de un FQDN

Cuando un cliente SOCKS5 solicita un destino por nombre de dominio, `getaddrinfo()` puede devolver más de una dirección. Por ejemplo, un mismo FQDN puede resolver a varias IPv4, varias IPv6 o una combinación de ambas. Un problema de implementación fue decidir qué hacer si la primera dirección devuelta no conectaba.

Una solución simple hubiera sido intentar solo la primera dirección, pero eso hacía al proxy menos robusto: podía fallar aunque otra dirección del mismo dominio estuviera disponible. Por eso se implementó un fallback secuencial. El servidor conserva la lista de direcciones retornada por `getaddrinfo()` y prueba conectarse a una dirección por vez. Si `connect()` falla, o si el socket no bloqueante termina con error, se avanza al siguiente candidato. Solo si se agotan todas las direcciones se informa el error al cliente.

Se eligió un fallback secuencial y no uno paralelo para mantener baja la complejidad del servidor. Una estrategia paralela, similar a Happy Eyeballs, podría reducir la latencia en algunos casos, pero implicaría manejar varios sockets salientes simultáneos por una misma conexión SOCKS5, cancelar los intentos perdedores y sincronizar más estados internos. Para el alcance del trabajo, el enfoque secuencial mejora la robustez sin complicar excesivamente el diseño.

El costo de esta decisión es que, si una dirección tarda en fallar, la conexión final puede demorarse más que con intentos paralelos. Por eso queda identificado como una posible mejora futura.

## 3. Limitaciones de la aplicación

- El protocolo de management no tiene autenticación propia.
- Las passwords de usuarios SOCKS5 se almacenan en texto plano en memoria.
- Solo se soporta `CONNECT`; no se implementan `BIND` ni `UDP ASSOCIATE`.
- La respuesta de `CONNECT` informa `BND.ADDR/BND.PORT` como `0.0.0.0:0`.
- La tabla de usuarios tiene capacidad fija de 10 entradas.
- El fallback multi-IP es secuencial, no paralelo.
- No hay disectores de protocolo de aplicación ni registro de credenciales POP3/ettercap-like en esta entrega.
- El servidor actual prioriza `USERNAME/PASSWORD` si el cliente ofrece ese método, aun cuando no haya usuarios configurados; si el cliente también ofrece `NO AUTH`, sería más consistente preferir `NO AUTH` en ese caso.
- `access.log` se abre, escribe y cierra en cada evento.
- No hay tests automatizados de integración SOCKS5/management; la suite actual cubre utilidades genéricas.

## 4. Posibles extensiones

- Agregar autenticación al protocolo de management.
- Soportar `BIND` y/o `UDP ASSOCIATE`.
- Resolver direcciones en paralelo al estilo Happy Eyeballs.
- Persistir usuarios a disco.
- Exponer métricas en JSON o formato compatible con Prometheus.
- Rotar `access.log` por tamaño o fecha.

## 5. Conclusiones

El desarrollo de este trabajo nos permitió ejercitar de forma concreta los tres objetivos centrales de la materia: la programación de aplicaciones cliente/servidor con sockets, la comprensión de un estándar de industria (SOCKS5, RFC 1928/1929) implementado desde cero, y el diseño de un protocolo de aplicación propio documentado de forma independiente de su implementación.

La restricción de operar con I/O no bloqueante multiplexada en un único hilo — salvo por la resolución DNS — resultó ser la tensión central que guió la mayoría de las decisiones de diseño del proyecto: desde la necesidad de una máquina de estados explícita tolerante a lecturas parciales de TCP, hasta el mecanismo de señales para despertar `pselect()` desde el hilo secundario de resolución de nombres. Esta misma restricción explica, en última instancia, el límite de escalabilidad que medimos empíricamente: el servidor sostiene un techo estable de 509 conexiones concurrentes, que coincide casi exactamente con el límite teórico impuesto por `FD_SETSIZE` bajo `select()/pselect()` una vez contabilizados los dos file descriptors que consume cada conexión en relay. Verificar esto con una prueba de estrés propia nos permitió confirmar en la práctica una consecuencia directa de una decisión de diseño tomada al principio del trabajo, y entender mejor por qué la consigna pide soportar "al menos 500" conexiones simultáneas.

## 6. Ejemplos de prueba

Compilar:

```
make clean
make all
```

Levantar servidor:

```
./bin/server -l 127.0.0.1 -p 1089 -L 127.0.0.1 -P 8089
```

Agregar usuario:

```
./bin/client -P 8089 ADDUSER alice secret
```

Probar management:

```
./bin/client -P 8089 STATS
./bin/client -P 8089 USERS
./bin/client -P 8089 HELP
```

Probar SOCKS5 con autenticación:

```
curl -x socks5h://alice:secret@127.0.0.1:1089 http://example.com
```

Probar SOCKS5 sin autenticación, si no hay usuarios configurados:

```
curl -x socks5h://127.0.0.1:1089 http://example.com
```

Revisar access log:

```
cat access.log
```

También se probó el comportamiento del servidor con muchas conexiones simultáneas. Para eso se levantó un servidor HTTP local como destino y se ejecutaron múltiples clientes curl en paralelo pasando por el proxy SOCKS5.

Servidor destino:

```
python3 -m http.server 29201 --bind 127.0.0.1
```

Servidor SOCKS5:

```
./bin/server -l 127.0.0.1 -p 18080 -L 127.0.0.1 -P 18081 -u alice:secret
```

Prueba con N conexiones concurrentes:

```
for i in $(seq 1 500); do
```

```
curl -s -x socks5h://alice:secret@127.0.0.1:18080 \
http://127.0.0.1:29201/ > /dev/null &
done
wait
```

Luego se consultaron las métricas:

```
./bin/client -P 18081 STATS
```

En las pruebas realizadas, el servidor pudo sostener alrededor de 509 conexiones SOCKS5 concurrentes. Por debajo de ese valor las conexiones se establecieron correctamente; por encima, el número de conexiones servidas con éxito se mantuvo alrededor de ese límite y el resto de los intentos falló al conectar.

Este resultado es consistente con la arquitectura elegida. El servidor usa `select()/pselect()`, por lo que queda limitado por `FD_SETSIZE`. Además, cada conexión SOCKS5 activa usa al menos dos file descriptors: uno para el cliente y otro para el destino. Descontando los sockets pasivos del servidor, el límite teórico queda cerca de:

$$(1024 - 2) / 2 = 511 \text{ conexiones}$$

El valor medido, cercano a 509, coincide con ese orden de magnitud. Al superar ese límite no se observó una caída gradual del rendimiento de cada conexión, sino un aumento de conexiones rechazadas: las conexiones que entraban seguían transfiriendo, pero cada vez más intentos nuevos quedaban afuera.

## 7. Guía de instalación detallada y precisa

Requisitos:

- sistema POSIX con sockets BSD;
- `gcc`;
- `make`;
- `pthread`;
- `python3` y `nc` para pruebas manuales;
- librería Check solo para tests unitarios.

Compilación:

```
git clone <url-del-repositorio>
cd Protos-TPE-G15
make all
```

Esto genera:

- `bin/server;`
- `bin/client.`

Tests:

```
make tests
```

El target genera un binario por cada archivo `tests/*.c` en `bin/`, si la librería Check está disponible.

Limpieza:

```
make clean
```

## 8. Instrucciones para la configuración

El servidor se configura por línea de comandos al arrancar y por management en runtime para altas/bajas de usuarios.

Ejemplo:

```
./bin/server -l 0.0.0.0 -p 1080 -L 127.0.0.1 -P 8080 -u pablito:pass1234
```

## 9. Ejemplos de configuración y monitoreo

Consultar métricas:

```
./bin/client STATS
```

Salida esperada:

```
historical_connections=42
concurrent_connections=3
successful_connections=39
failed_connections=3
bytes_transferred=1048576
bytes_client_to_target=524288
bytes_target_to_client=524288
```

Listar usuarios:

```
./bin/client USERS
```

Agregar usuario:

```
./bin/client ADDUSER pablito pass1234
```

Eliminar usuario:

```
./bin/client DELUSER pablito
```

Ver comandos:

```
./bin/client HELP
```

## 10. Documento de diseño del proyecto

Estructura del repositorio:

src/

server/

|                        |                                                                                               |
|------------------------|-----------------------------------------------------------------------------------------------|
| server.c/.h            | arranque del servidor, sockets pasivos, selector, señales y ciclo principal                   |
| client_session.c/.h    | aceptación de clientes SOCKS5, estados GREETING, AUTH, REQUEST, RELAY y cierre                |
| target_connection.c/.h | resolución DNS, conexión no bloqueante al destino, fallback multi-IP y handlers del target_fd |
| management.c/.h        | protocolo de management y socket pasivo de administración                                     |
| auth.c/.h              | autenticación RFC1929                                                                         |
| socks5.c/.h            | constantes, validaciones y armado de respuestas SOCKS5                                        |
| users.c/.h             | tabla de usuarios SOCKS5 en memoria                                                           |
| metrics.c/.h           | contadores de métricas                                                                        |
| access_log.c/.h        | registro de accesos                                                                           |

client/

|          |                       |
|----------|-----------------------|
| client.c | cliente de management |
|----------|-----------------------|

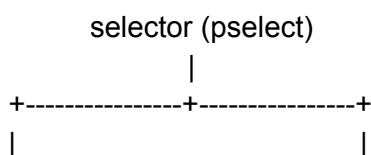
utils/

|                   |                                      |
|-------------------|--------------------------------------|
| selector.c/.h     | multiplexor de I/O no bloqueante     |
| stm.c/.h          | motor genérico de máquinas de estado |
| buffer.c/.h       | buffer genérico                      |
| parser.c/.h       | parser genérico                      |
| parser_utils.c/.h | utilidades de parser                 |
| netutils.c/.h     | utilidades de red                    |
| args.c/.h         | parseo de argumentos                 |

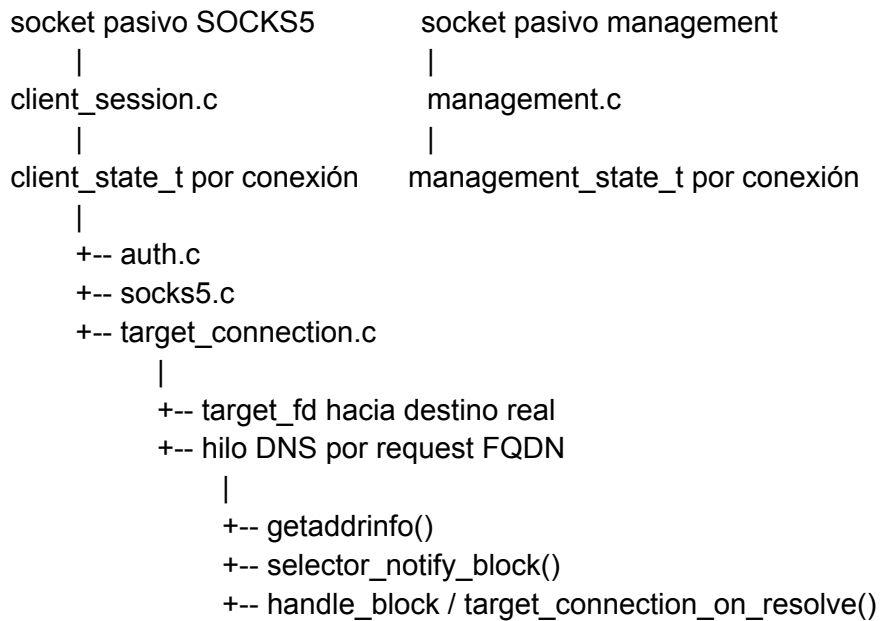
tests/

tests unitarios sobre utilidades

Vista de componentes en runtime:







Relay:

cliente <-> client\_session.c <-> target\_connection.c <-> destino

Hilo DNS por request FQDN:

getaddrinfo() -> selector\_notify\_block() -> handle\_block

Decisiones de diseño relevantes:

- máquina de estados manual para SOCKS5, por simplicidad y control explícito sobre lecturas parciales
- separación modular entre server.c, client\_session.c, target\_connection.c, management.c y auth.c para reducir el tamaño y responsabilidad de cada archivo
- un hilo solo para resolución DNS, no para I/O de datos
- conexión saliente no bloqueante usando EINPROGRESS y verificación posterior con getsockopt(SO\_ERROR)
- protocolo de management textual, simple y no persistente
- management por defecto en loopback
- buffers separados por sentido para el relay